

Docker & Containers

Module 6 — Cybersecurity Fundamentals

Today

- Docker lecture — 60 minutes
- Docker lab — 30 minutes (pre-pull the Day 5 images)
- Then the Day 5 labs — Splunk and Volatility
- Goal: run any security tool that ships as a container
- You already know the Linux shell from this morning

Why Containers

"It works on my machine"

- Software needs a specific version of libraries and tools
- Your machine and a server rarely match exactly
- A tool that runs for you may crash for a teammate
- Containers package the app with everything it needs
- Same package runs the same way everywhere

What is a container

- A lightweight, isolated box for running one application
- Has its own filesystem, processes, and network view
- Shares the host's Linux kernel — nothing extra to boot
- Starts in seconds; uses little memory and disk
- When you delete it, it leaves no trace behind

Container vs virtual machine

- A VM emulates whole hardware and runs a full guest OS
- A VM is gigabytes in size and boots in minutes
- A container shares the host kernel — no guest OS
- A container is megabytes and starts in seconds
 - VM = stronger isolation; container = far lighter

Image vs container

- An image is the read-only template — the app plus its files
- A container is a running instance created from an image
- One image can start many containers at once
- Analogy: image is the recipe, container is the cooked meal
- `docker run nginx`
 - Above: one image, starts one container

Docker architecture and Docker Hub

- The Docker CLI is the command you type
- The Docker daemon is the background service doing the work
- The CLI sends your commands to the daemon
- A registry is an online store of images
- Docker Hub is the default public registry
 - Security tools like Splunk publish images there

Running Containers

Your first container

- `docker run hello-world`
- Docker pulls the image if it is not already local
- Then it creates and starts a container from it
- The container prints a message and exits
- One command: download, create, run

Listing containers and images

- `docker ps`
- Shows containers that are currently running
- `docker ps -a`
- Shows all containers, including stopped ones
- `docker images`
- Lists the images stored locally on your machine

The container lifecycle

- States: created, running, stopped, removed
- `docker stop <name>`
- `docker start <name>`
- `docker rm <name>`
- Stop pauses it; start resumes; rm deletes it for good

Naming and detached mode

- Without a name Docker assigns a random one
- `docker run --name web -d nginx`
 - `--name`
 - gives the container a name you choose
 - `-d`
 - runs it in the background (detached)

Getting a shell inside a container

- `exec`
- Runs a new command inside a running container
- `docker exec -it web bash`
 - `-it`
 - makes the session interactive with a terminal
- Type `exit` to leave the container, not to stop it

Reading container logs

- Logs are whatever the app prints to the screen
- `docker logs web`
- Shows all output the container has produced
- `docker logs -f web`
 - `-f`
 - follows new output live, like tail -f

Publishing ports

- A container's network is isolated by default
- Port publishing connects a host port to a container port
- `docker run -d -p 8000:80 nginx`
 - Format is -p HOST:CONTAINER
- Then open `http://localhost:8000` in a browser
- Without -p the service is unreachable from the host

Running containers — recap

- `docker run`
 - create and start a container
- `docker ps / logs / exec`
 - inspect a running container
- `docker stop / rm`
 - stop and delete when done

Images & Data



Pulling images

- `docker pull ubuntu`
- Downloads an image from a registry without running it
- A tag picks a specific version after a colon
- `docker pull splunk/splunk:latest`
 - latest just means the default tag, not newest-safe

Where images come from — the Dockerfile

- A Dockerfile is a recipe for building an image
- Each line is a step: base image, copy files, install tools
- `FROM ubuntu:22.04`
- `RUN apt-get install -y curl`
- You will use ready-made images today, not build your own

Image layers

- Each Dockerfile step adds one read-only layer
- Layers stack to form the final image
- Shared layers are stored once and reused across images
- This makes pulls and builds fast and disk-efficient
- See the image-layers-explorer demo

Containers are ephemeral

- A container gets its own writable layer on top of the image
- Files you create live only in that container
 - `docker rm`
 - deletes the container and all its changes
- Ephemeral = temporary; nothing inside survives removal
- To keep data you must store it outside the container

Volumes and bind mounts

- Both keep data alive outside the container
- A volume is storage Docker manages for you
- `docker run -v mydata:/data ubuntu`
- A bind mount maps a real folder on your host
- `docker run -v /home/kali/files:/data ubuntu`
 - Bind mounts are great for feeding evidence files in

Docker Compose

Why Docker Compose

- Real tools often need several containers together
- Example: an app container plus a database container
- Long docker run commands are easy to get wrong
- Compose defines the whole setup in one file
- One command starts or stops everything at once

The compose.yaml file

- A plain text file listing each container as a service
- Per service: image, ports, volumes, environment
- services: web uses nginx, db uses postgres
 - `web:`
 - `image: nginx`
 - `ports: ["8080:80"]`

Running Compose

- `docker compose up -d`
- Starts every service in the file, in the background
- `docker compose down`
- Stops and removes all of them cleanly
- Run both from the folder holding `compose.yaml`

Windows Containers

Windows containers exist

- Containers can also package Windows applications
- Used for older .NET apps and IIS web workloads
- They require a Windows host to run on
- A Linux host cannot run Windows containers
- Almost all security tooling uses Linux containers

Out of scope today

- We will not run any Windows containers in this course
- Every Day 5 lab image is a Linux container
- Splunk and Volatility both run on Linux
- Everything else in this deck assumes Linux containers
- If you meet a Windows container later, the concepts still apply

Security: Use & Risk

Containers as a disposable sandbox

- Spin up a clean, isolated environment in seconds
- Analyze a suspicious file or tool inside it
- `docker run --rm -it ubuntu bash`
 - `--rm`
 - auto-deletes the container when you exit
- Throw it away and the host stays untouched

The daemon runs as root

- The Docker daemon runs with full root privileges
- Anyone who can run docker can effectively become root
- `docker run -v /:/host -it ubuntu`
 - That mounts the whole host filesystem inside
- Treat docker access as administrator access

--privileged and poisoned images

- `--privileged`
- Drops most isolation — the container can touch the host
- Avoid it unless a tool truly demands it
- Public images can contain malware or backdoors
- Prefer official or verified publishers; check the tag
 - Pin a real version instead of latest

Clean up after yourself

- Stopped containers and unused images pile up on disk
- Old containers can also be a forgotten attack surface
- `docker stop $(docker ps -q)`
- `docker system prune`
 - prune removes stopped containers and unused data

Summary

- A container packages an app so it runs anywhere
- Image is the template; container is the running instance
- **Daily workflow: run, ps, logs, exec, stop, rm**
- Use volumes or bind mounts to keep data
- Compose runs multi-container setups from one file
- Treat docker access as root — trust your images

Thank you